

Phần 1: Xác định vi phạm phong cách lập trình tốt Trong các đoạn chương trình sau, hãy chỉ ra vi phạm (nếu có) về phong cách lập trình tốt. Sử dụng các lựa chọn sau trong câu trả lời:

(a) Tính tổng quát

(b) Hiệu quả

(c) Định dạng

(d) Tài liệu hóa (ghi chú)

(e) Không vi phạm

câu 1: đã vi phạm nguyên tắc về tính tổng quát và tài liệu hóa vì đoạn chương trình trên bị áp vào parts.dat nếu muốn đọc file khác thì phải đổi tên, còn vi phạm về tài liệu hóa là vì nó chỉ giải thích chương trình làm gì chứ không giải thích tại sao phải làm như thế

câu 2: đã vi phạm nguyên tắc về tính hiệu quả và tài liệu hóa vì đoạn chương trình đã chạy hai lần trên cùng 1 danh sách khiến time complexity của nó là $O(n^2)$ trong khi ta có thể chỉ cần triển khai với đúng duy nhất 1 vòng lặp for

câu 3: đã vi phạm nguyên tắc về tính hiệu quả và tổng quát vì giá trị MAXSALES của đoạn chương trình trên luôn cố định là 100000 tức là nó sẽ luôn cố định với giá trị đó cho dù là 5 hay 100 giao dịch điều này dẫn đến lãng phí tài nguyên trong khi ta chỉ cần dùng hàm cấp phát malloc cho giá trị sales là vẫn mang lại hiệu quả tương tự mà không lãng phí tài nguyên máy, bên cạnh đó hàm này chỉ dùng để tính tổng nếu sau này cần tính trung bình, min hoặc max thì phải thêm hàm mới thay vào có thể tổng quát hóa hơn bằng cách truyền thêm tham số kiểu tính toán, hoặc tách dữ liệu ra để tái sử dụng.

câu 4: Chương trình này không vi phạm về phong cách lập trình tốt

câu 5: đã vi phạm nguyên tắc về tính định dạng và tài liệu hóa vì đoạn chương trình này vẫn chỉ mô tả nó làm gì chứ không ghi tại sao lại làm như vậy, bên cạnh đó chương trình cũng vi phạm định dạng khi vòng lặp for bị thiếu dấu `{}`.

Phần 2: Phân loại lỗi Trong quá trình rà soát mã nguồn, các lỗi sau được phát hiện. Hãy phân loại kiểu lỗi trong mỗi đoạn mã.

câu 6: đáp án là D, chỉ b và c.

câu 7: Đáp án là C, lỗi chính xác `int i nên = 0;`

câu 8: Đáp án là C, lỗi quá tải / vượt dung lượng vì nếu $i \leq 10$ thì khi đó `list[10] = 10` dẫn đến buffer overflow

câu 9: Đáp án là D, a, b và c